

ML Engineer — Easy Guide

Crash course before the presentation · Retail Demand Forecasting (Walmart M5)

In one sentence

We train **LightGBM** — lots of small decision trees — with a **Tweedie** objective, because 68% of our sales values are exactly zero.

Machine learning in 30 seconds

Normally a programmer writes the rules. In **ML (Machine Learning)** the computer finds the rules from examples.

- **Situation** = 38 numbers about a product on a day (the features)
- **Answer** = how many units it actually sold
- **Learning** = adjusting the model until its guesses get close
- **Using it** = feed in a future situation, read off the guess

***AI vs ML.** AI is the broad field. ML is the part that learns from data. Our forecasting model is ML. Gemini is a *different* AI that only explains our numbers — it never produces them.*

LightGBM, built up in three steps

1. A decision tree — a flowchart of yes/no questions:

```
is rolling_mean_28 < 0.2 ?
  yes -> is days_since_last_sale > 30 ?
          yes -> predict 0.02
          no  -> predict 0.15
  no  -> predict 2.1
```

2. Boosting — one tree is weak. Build a tree, look at where it's wrong, build a second tree that predicts *the remaining error*, then a third. Add them up.

3. "Light" — LightGBM is engineered to do this fast on tens of millions of rows, and it handles categories like `store_id` natively.

LightGBM = Light Gradient Boosting Machine.

Tweedie — the most important choice we made

***Objective (loss function)** = the definition of "wrong" that the model tries to minimise.*

The default is squared error, which assumes the target is roughly a bell curve. **Ours is nothing like a bell curve** — 68% of values are exactly 0.

Tweedie is built for exactly this shape: a big lump at zero plus a positive tail. It's what insurance uses for claims — most policies claim nothing, a few claim a lot. Retail demand behaves the same way.

```
tweedie_variance_power = 1.1
```

- near **1** = Poisson-like (counts)
- near **2** = Gamma-like (positive, no zeros)
- **1.1** = mostly-small counts with a huge mass at zero ✓

Why predictions look like 0.6: Tweedie predicts an **expected value**. 0.6 units/day is about 17 units over 28 days. That's a real number, not a bug.

Our two models

	Direct	Recursive
Features	38	32
How it predicts	all 28 days at once	one day, fed back in, 28 times
Training design	15 origins $\times$ 30,490 $\times$ 28 = 12,805,800 rows	420 daily origins, d_1521...d_1940
RMSE alone	2.1211	2.1185

Blended $0.60 \times \text{direct} + 0.40 \times \text{recursive}$ $\rightarrow$ RMSE 2.0929 — better than either.

Both are **global** models: one model for all 30,490 series, not 30,490 models. With most products selling rarely, per-series models would starve for data.

Settings worth knowing

Setting	Value	Plain English
Boosting rounds	400	how many trees
Learning rate	0.05	how big a step each tree takes
num_leaves	128	how complex each tree may be
min_data_in_leaf	100	a rule needs ≥ 100 examples behind it
feature_fraction / bagging_fraction	0.8 / 0.8	each tree sees 80% of features/rows
lambda_l2	1.0	penalty against extreme predictions
Seed	42, deterministic	same run $\rightarrow$ same model, always

Overfitting = memorising the training data instead of learning the pattern. The last four settings all exist to prevent it.

The split is by TIME, never random

Block	Days
Training	d_1 ... d_1913 (to 2016-04-24)
Validation	d_1914 ... d_1941 (2016-04-25 $\rightarrow$ 05-22)
Final forecast	d_1942 ... d_1969 (2016-05-23 $\rightarrow$ 06-19)

A random split would train on the future and test on the past — meaningless for forecasting.

Numbers to memorise

RMSE (blend)	2.0929
MAE (blend)	1.0395
Predictions evaluated	853,720
Direct / recursive alone	2.1211 / 2.1185
Naive baseline	2.2430
Predict-zero baseline	3.9161

Experiments recorded	86
Residual correlation between members	0.9488

The frozen model

Frozen = final. Never retrained, re-tuned or replaced.

Why? Every accuracy number the app shows was measured against **these exact weights**. Retrain, and every number on every page describes a model that no longer exists.

How it's enforced:

- CHAMPION_MANIFEST.json stores the SHA-256 hash of both model files
- test_integrity.py re-hashes them and fails if they changed
- model files are mounted **read-only**; the API runs as a non-root user
- 522 protected files tracked; last check: **0 deleted, 0 modified**

Model files: model_11_blend_direct_final_forecast.txt, model_12_blend_recursive_shape_final.txt in 03_RESEARCH/models/champion/.

Training vs inference

	Training	Inference
What	Learns from known answers	Applies the saved model
When	Once, in research	Whenever a forecast is needed
Output	A model file	Predictions

Verified timings: direct member over all 853,720 rows = **4.1 s**; recursive rollout $\approx$ **29 s**; full blended re-forecast $\approx$ **33 s**.

The proof that it still works

POST /api/v1/inference/verify reloads both frozen models, rebuilds features, re-runs the blend, and compares to the shipped forecast.

VERIFIED: max_abs_diff = 0.0000e+00 across all 853,720 predictions.

Why we stopped at 2.0929

Three independent pieces of evidence say RMSE below 2.0 isn't reachable **with this data**:

- Six architecturally different models produce residuals correlated at **0.9897** — they make the same mistakes.
- Every feature family was tested; none helped further.
- An oracle that already knows each series' true 28-day mean only gets **1.9818**.

The limit is **information**, not modelling. What would help is a promotions calendar or inventory data — neither exists in this dataset.

How to explain it to a judge

"My part is the model. We use LightGBM, which builds lots of small decision trees where each tree corrects the errors of the previous ones.

The key decision is the objective — the definition of 'wrong'. We use Tweedie with variance power 1.1, because 68% of our data is exactly zero. Standard squared error assumes a bell curve; our data isn't remotely that shape. Tweedie handles a big spike at zero plus a positive tail — the same thing insurance uses for claims.

We train two models: a direct one that predicts all 28 days at once, and a recursive one that predicts a day and feeds it back in. Alone they're 2.1211 and 2.1185 RMSE; blended 60/40 we get 2.0929 — better than either, because they're different architectures and make different mistakes.

It's all seeded and deterministic, and the model is frozen — we store hashes of both model files and a test fails if they change. And we can prove it: an endpoint reloads the models and re-runs the whole forecast, matching the shipped numbers with a maximum absolute difference of exactly zero."

Quick answers

Why Tweedie? 68% zeros. Squared error assumes a bell curve; Tweedie models a spike at zero plus a positive tail. *Why one global model?* 30,490 series, most selling rarely — per-series models would have far too little data. *Why is 0.6 a valid prediction?* It's an expected value — ~17 units over 28 days. *How do you stop overfitting?* min_data_in_leaf 100, 80% feature/row sampling, L2 penalty, lr 0.05, and a time-based split. *Could you do better than 2.0929?* Not with this data — the oracle bound is 1.9818 and six architectures correlate at 0.9897. *Does the AI assistant make the forecast?* No. LightGBM does. Gemini only explains it.

30-second revision

LightGBM · Tweedie 1.1 · 400 rounds · lr 0.05 · 128 leaves · seed 42 deterministic · two members (38 features direct / 32 recursive) · blend 0.60/0.40 · RMSE **2.0929**, MAE **1.0395** on **853,720** predictions · 86 experiments · frozen and hash-verified · re-inference max_abs_diff = 0.000e+00.